

EXPERIMENT-5

Implement the K-Nearest Neighbors (KNN) algorithm in Python.

Code:

```
import pandas as pd
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix

print("="*70)
print("      K-NEAREST NEIGHBORS (KNN)")
print("="*70)

uploaded = files.upload()
data = pd.read_csv("Iris.csv")

print("\nDataset\n")
print(data.head(10))

X = data.iloc[:,1:5]
y = data.iloc[:,5]
encoder = LabelEncoder()
y = encoder.fit_transform(y)
X_train,X_test,y_train,y_test=train_test_split(
X,y,test_size=0.3,random_state=42)
```

```

print("\nTraining Samples :",len(X_train))
print("Testing Samples :",len(X_test))

model=KNeighborsClassifier(n_neighbors=3)

print("\nTraining KNN Model...")

model.fit(X_train,y_train)

prediction=model.predict(X_test)

print("\nActual Values")
print(y_test)
print("\nPredicted Values")
print(prediction)
print("\nAccuracy")
print(round(accuracy_score(y_test,prediction)*100,2),"%")
print("\nConfusion Matrix")
print(confusion_matrix(y_test,prediction))

sample=[[5.1,3.5,1.4,0.2]]
result=model.predict(sample)
print("\nNew Sample :",sample)
print("Predicted Class :",encoder.inverse_transform(result))

```

Sample Input:

```

...  =====
      K-NEAREST NEIGHBORS (KNN)
      =====
      Choose Files Iris.csv
      Iris.csv(text/csv) - 5107 bytes, last modified: 8/5/2026 - 100% done
      Saving Iris.csv to Iris (1).csv

```

Sample Output:

... Dataset

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
5	6	5.4	3.9	1.7	0.4	Iris-setosa
6	7	4.6	3.4	1.4	0.3	Iris-setosa
7	8	5.0	3.4	1.5	0.2	Iris-setosa
8	9	4.4	2.9	1.4	0.2	Iris-setosa
9	10	4.9	3.1	1.5	0.1	Iris-setosa

Training Samples : 105

Testing Samples : 45

Training KNN Model...

Actual Values

```
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]
```

Predicted Values

```
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]
```

Accuracy

100.0 %

Confusion Matrix

```
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

New Sample : [[5.1, 3.5, 1.4, 0.2]]

Predicted Class : ['Iris-setosa']